

COMPUTER ORGANIZATION & OPERATING SYSTEMS (COOS)

PRACTICAL & ORAL (VIVA VOCE) MASTER GUIDE

Target Experiments: Exp 2, 4, 5, 6, 8, 9, 10, 11

Candidate: Aaryan Choube (24CE1045) | Batch: A / A1 | RAIT, DY Patil University

Computer Organization & Operating Systems (COOS) Master Viva & Practical Guide

Prepared for Practical & Oral (Viva Voce) Examination

Target Experiments: Exp 2, Exp 4, Exp 5, Exp 6, Exp 8, Exp 9, Exp 10, and Exp 11

Candidate: Aaryan Choube (24CE1045) | **Batch:** A / A1

Department: Computer Engineering, RAIT, DY Patil University

⚡ EXPERIMENT 2: COMPUTER ARITHMETIC ALGORITHMS

Q1: What is Booth's Multiplication Algorithm? Why is it preferred?

Ans: Booth's algorithm is an efficient hardware multiplication algorithm that multiplies two signed binary numbers in 2's complement representation.

- Key Advantage:** It handles both positive and negative numbers uniformly without requiring separate sign-magnitude conversion.
- It speeds up multiplication by replacing blocks of consecutive 1s with a single addition and subtraction:

$A \times (0111110)_2 = A \times (2^6 - 2^1)$

Instead of 5 additions, it performs only 1 subtraction and 1 addition!

Q2: Explain the decision rules of Booth's Algorithm ($Q_0 Q_{-1}$).

Ans: At each step, inspect the last bit of multiplier Q_0 and the previous shifted-out bit (Q_{-1}):

- 10:** Subtract Multiplicand from Accumulator ($A = A - M$), then Arithmetic Right Shift $[A, Q, Q_{-1}]$.
 - 01:** Add Multiplicand to Accumulator ($A = A + M$), then Arithmetic Right Shift $[A, Q, Q_{-1}]$.
 - 00 or 11:** Do nothing to A , just Arithmetic Right Shift $[A, Q, Q_{-1}]$.
- Repeat for n cycles (where n is bit-length). The final $2n$ -bit product is in $[A, Q]$.

Q3: What is Arithmetic Right Shift (ASR) and why is it used?

Ans: In Arithmetic Right Shift, all bits shift right by 1, and the **Most Significant Bit (MSB / sign bit)** is retained/copied into the new MSB position ($A[n-1] = A[n-1]$). This preserves the arithmetic sign of negative numbers in 2's complement.

Q4: Compare Restoring and Non-Restoring Division Algorithms.

Feature | Restoring Division | Non-Restoring Division

****Basic Operation**** | Always subtracts: $A = A - M$ | If $A \geq 0$ implies $A = A - M$; If $A < 0$ implies $A = A + M$

****Restoration Step**** | If $A < 0$, must restore $A = A + M$ and set $Q_0 = 0$. | ****No restoration**** during cycles. Directly sets $Q_0 = 0$ or 1 .

****Clock Cycles**** | Takes extra clock cycle whenever $A < 0$ (slower). | Takes exactly 1 add/sub operation per bit cycle (faster).

****End Correction**** | Not required. | Required: If final remainder $A < 0$, restore $A = A + M$.

****Register Output**** | Quotient in Q , Remainder in A . | Quotient in Q , Remainder in A .

EXPERIMENT 4: LINUX COMMANDS & SHELL SCRIPTING

Q5: What is the difference between a System Call and a Library Function?

****Ans:****

- ****System Call:**** Programmatic interface to request services directly from the OS kernel (e.g., `open`, `read`, `write`, `fork`). Executes in ****Kernel Mode / Privileged Mode**** via a hardware trap/interrupt.
- ****Library Function:**** Pre-compiled subroutine provided by the runtime/C standard library (e.g., `printf`, `strcpy`, `fopen`). Executes in ****User Mode****; may internally invoke one or more system calls (e.g., `printf` invokes `write`).

Q6: Explain file permissions in Linux (`chmod 754 file.txt`).

****Ans:**** Permissions are represented by octal digits for Owner, Group, and Others:

- Read (r) = 4, Write (w) = 2, Execute (x) = 1.
- In `754`:
- ****7 (\$4+2+1\$):**** Owner has Read, Write, and Execute (`rwX`).
- ****5 (\$4+0+1\$):**** Group has Read and Execute (`r-x`).
- ****4 (\$4+0+0\$):**** Others have Read-only (`r--`).

Q7: Explain commands for the 5 teacher-specified requirements:

1. ****OS & Kernel Info:**** `uname -a`, `uname -r` (kernel release), `cat /etc/os-release`.
2. ****Top 10 CPU Processes:**** `ps -eo pid,ppid,user,%cpu,comm --sort=-%cpu | head -n 11`.
3. ****Top 10 Memory Processes:**** `ps -eo pid,user,%mem,comm --sort=-%mem | head -n 11`.
4. ****Current User & Log Name:**** `whoami`, `logname`, `who`.
5. ****Environment & Shell:**** `echo $SHELL`, `echo $HOME`, `echo $OSTYPE`, `pwd`, `echo $PATH`.

EXPERIMENT 5: LINUX KERNEL APIS (`cp`, `ls`, `mv`)

Q8: How is the `cp` command implemented using Linux system calls?

****Ans:****

1. Open source file using `open(src, O_RDONLY)`.
2. Open/create destination file using `open(dest, O_WRONLY | O_CREAT | O_TRUNC, 0644)`.
3. Loop: Read data chunks using `read(src_fd, buffer, BUF_SIZE)`.

4. Write read bytes using `write(dest_fd, buffer, bytes_read)`.
5. Close file descriptors using `close(src_fd)` and `close(dest_fd)`.

Q9: How is the `ls` command implemented using Directory APIs?

Ans:

1. `opendir(path)`: Opens the directory stream and returns a `DIR*` pointer.
2. `readdir(dir_ptr)`: Reads directory entries one by one, returning a pointer to `struct dirent`.
 - `struct dirent` contains `d_name` (filename), `d_type` (file type), `d_ino` (inode number).
3. `closedir(dir_ptr)`: Closes the directory stream.

Q10: How is the `mv` command implemented?

Ans:

- Uses the `rename(old_path, new_path)` system call.
- If across different filesystems, it copies the file (`open` + `read` + `write`) and then deletes the original using `unlink(old_path)`.

EXPERIMENT 6: LINUX PROCESS CREATION (fork, orphan, zombie)

Q11: What is the return value of `fork()` system call?

Ans:

- **Negative value (<0):** Process creation failed.
- **Zero (0):** Returned to the newly created **Child process**.
- **Positive value (>0):** Returned to the **Parent process** (value is the PID of the child).

Q12: How many processes are created if `fork()` is called k times consecutively?

Ans:

- Total number of processes = 2^k (including parent).
- Total number of **child** processes = $2^k - 1$.
- **Example:** If `fork()` is called 3 times $\implies 2^3 = 8$ processes total (1 parent + 7 children).

Q13: What is a Zombie Process? How is it resolved?

Ans:

- **Definition:** A process that has finished execution (called `exit()`), but its entry remains in the OS Process Table because its parent has not yet read its exit status via `wait()` or `waitpid()`.
- **Status in `ps`:** Marked as `Z` or `<defunct>`.
- **Dangers:** It consumes a Process Control Block (PCB) and PID slot. If too many accumulate, no new processes can be created.
- **Prevention:** Parent must call `wait(NULL)` or handle `SIGCHLD` signal.

Q14: What is an Orphan Process? Who adopts it?

Ans:

- **Definition:** A child process whose parent terminates before the child completes.

- **Adoption:** The orphan process is immediately adopted by the root init process ('init' in SysV or 'systemd' in modern Linux, with **PID 1**). Init automatically reaps the child when it terminates.

EXPERIMENT 8: DEADLOCK AVOIDANCE (BANKER'S ALGORITHM)

Q15: What are Coffman's 4 Necessary Conditions for Deadlock?

Ans: Deadlock can occur if and only if all 4 conditions hold simultaneously:

1. **Mutual Exclusion:** At least one non-shareable resource held by a process.
2. **Hold and Wait:** A process holding a resource is waiting for additional resources held by others.
3. **No Preemption:** Resources cannot be forcibly confiscated; they are released voluntarily.
4. **Circular Wait:** A closed chain of processes $\{P_0, P_1, \dots, P_n\}$ exists where each process waits for a resource held by the next.

Q16: Explain the Data Structures of Banker's Algorithm.

- n = Number of processes, m = Number of resource types.
- **Available[m]:** Number of available units of each resource type.
- **Max[n][m]:** Maximum demand matrix of each process.
- **Allocation[n][m]:** Resources currently allocated to each process.
- **Need[n][m]:** Remaining resources needed:

$$\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$$

Q17: What is the Safety Algorithm?

Ans:

1. Let $\text{Work} = \text{Available}$ and $\text{Finish}[i] = \text{false}$ for all i .
2. Find index i such that $\text{Finish}[i] == \text{false}$ and $\text{Need}_i \leq \text{Work}$.
3. If found: $\text{Work} = \text{Work} + \text{Allocation}_i$, $\text{Finish}[i] = \text{true}$, add P_i to Safe Sequence. Repeat step 2.
4. If all $\text{Finish}[i] == \text{true}$, the system is in a **Safe State**. If no such i exists and some $\text{Finish}[i] == \text{false}$, system is in an **Unsafe State** (risk of deadlock).

EXPERIMENT 9: DYNAMIC MEMORY PARTITIONING

Q18: Differentiate between Internal and External Fragmentation.

- **Internal Fragmentation:** Allocated memory block is slightly larger than the requested size; the unused space inside the partition is wasted.
- **External Fragmentation:** Total free memory exists to satisfy a request, but it is split into non-contiguous, scattered blocks and cannot be allocated. Solved by **Compaction** or **Paging**.

Q19: Compare First Fit, Best Fit, and Worst Fit.

Strategy | Search Rule | Speed | Fragment Left

First Fit | Allocates the **first hole** that is big enough. | **Fastest** (stops on first match). | Random leftover sizes.

Best Fit | Allocates the **smallest hole** that is big enough ($\min(\text{hole} - \text{process})$). | Slower (searches entire list). | Leaves **smallest leftover fragment** (often unusable).

Worst Fit | Allocates the **largest available hole** ($\max(\text{hole})$). | Slower (searches entire list). | Leaves **largest leftover fragment** (usable for other processes).

EXPERIMENT 10: VIRTUAL MEMORY PAGE REPLACEMENT

Q20: What is a Page Fault? How does the OS handle it?

Ans: A page fault occurs when a program accesses a virtual memory page that is not currently mapped into physical RAM (valid-invalid bit is 0).

Handling Steps:

1. CPU generates a trap/interrupt to the OS kernel.
2. OS checks internal tables to verify valid reference.
3. OS finds a free physical frame (or selects a victim page via page replacement).
4. OS schedules disk I/O to read the required page into the frame.
5. OS updates the page table (sets valid bit to 1).
6. Instruction is restarted.

Q21: What is Belady's Anomaly? Which algorithm exhibits it?

Ans:

- **Belady's Anomaly:** Counter-intuitive phenomenon where **increasing the number of page frames results in MORE page faults** for a given reference string!
- **Algorithm:** **FIFO (First-In, First-Out)** suffers from Belady's Anomaly.
- **Example Reference String:** 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5.
- With 3 frames: **9 page faults**.
- With 4 frames: **10 page faults** (faults increased!).

Q22: Why do LRU and Optimal NOT suffer from Belady's Anomaly?

Ans: LRU and Optimal belong to a class of algorithms called **Stack Algorithms**. For any stack algorithm, the set of pages in memory with n frames is always a subset of the pages in memory with $n+1$ frames:

$$S(n) \subseteq S(n+1)$$

Therefore, adding frames can never increase page faults.

Q23: Why cannot the Optimal algorithm be used in real operating systems?

Ans: The Optimal algorithm requires **future knowledge** of which pages will be referenced, which cannot be predicted in advance. It serves solely as an ideal theoretical benchmark to evaluate real algorithms.

EXPERIMENT 11: DISK SCHEDULING ALGORITHMS

Q24: What are the components of Disk Access Time?

Ans:

1. **Seek Time:** Time taken by the disk arm to move the read/write head to the desired cylinder/track. (Largest component!).
 2. **Rotational Latency:** Time taken for the target sector to rotate underneath the read/write head.
 3. **Transfer Time:** Time taken to read/write the actual data.
- **Disk Scheduling Goal:** Minimize total **Seek Time** (Total Head Movement).

Q25: Compare FCFS, SSTF, SCAN, and C-SCAN.

Algorithm | Movement Strategy | Advantages | Disadvantages

FCFS | Serves requests in arrival order. | Fair, simple, zero starvation. | High seek time, wild head swings.

SSTF | Serves request closest to current head position. | Much lower seek time than FCFS. | **Starvation** of distant requests!

SCAN (Elevator) | Moves in one direction to disk boundary, then reverses. | Low seek time, no starvation. | Inner/outer tracks wait longer than center.

C-SCAN (Circular) | Moves in one direction to end, jumps back to start without servicing. | **Uniform waiting time** across all cylinders. | Long return jump without servicing.

LOOK / C-LOOK | Same as SCAN/C-SCAN, but reverses at the **furthest request**, not the disk boundary. | Eliminates unnecessary trips to track 0 or 199. | Slightly more complex boundary check.